Studio Nx

백엔드 266 — 트랙 한 줄과 전체 조회

2026-10-19

Backend(FastAPI/Python) 266 — 트랙 한 줄과 전체 조회 (`_full_track` · `get_full_project`)

항목	내용
파일	`studionx/backend/app/services/full_project_service.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 전체 조회 서비스 (PRD 02 FR-254)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	265 `_group_by` · `project_info` · 267 `project_service.update_settings`

1. 하는 일

시퀀서가 화면을 열 때 부르는 한 번의 조회를 실제로 처리합니다.

```
get_full_project(db, user, project_id)
1. project_service.get_membership        멤버인지 확인 (아니면 404)
2. tracks       트랙 목록 (order_index 순)          조회 1번
3. instruments 악기 칸 (slot_index 순)              조회 1번 — 트랙 번호들로 한꺼번에
4. effects      이펙트 체인 (slot_index 순)          조회 1번
5. links        갈래 연결선                         조회 1번
6. patterns     패턴 목록                           조회 1번
7. notes        노트 (start_tick · pitch 순)         조회 1번
8. _group_by 로 트랙별 · 패턴별로 묶어 응답 조립
```

`_full_track` 은 그중 트랙 한 줄을 만드는 부분입니다.

2. 언제 쓰나

쓰는 곳	
`GET /projects/{id}/full` (268)	라우터가 이 함수를 그대로 부릅니다
시퀀서 화면 열기	트랙 · 악기 · 이펙트 · 패턴 · 노트를 한 번에 받아 그립니다 (FR-254)

3. 코드

```
def _full_track(track, instruments, effects, links) -> FullTrackResponse:
    """트랙 한 줄 + 그 트랙의 악기·이펙트·연결선 (FR-245 · 263 · 267 · 269)."""
    base = TrackResponse.model_validate(track).model_dump()
    return FullTrackResponse(
        **base,
        instruments=[TrackInstrumentResponse.model_validate(r) for r in instruments.get(track.track_id,
[])],
        effects=[TrackPluginResponse.model_validate(r) for r in effects.get(track.track_id, [])],
        links=[TrackPluginLinkResponse.model_validate(r) for r in links.get(track.track_id, [])],
    )
```

```
def get_full_project(db, user, project_id) -> FullProjectResponse:
    """프로젝트 전체 (FR-254). 멤버면 누구나 (VIEWER 포함)."""
    project, member = project_service.get_membership(db, project_id, user)

    tracks = list(db.scalars(
        select(Track).where(Track.project_id == project_id).order_by(Track.order_index, Track.track_id)
    ).all())
    track_ids = [t.track_id for t in tracks]
    if track_ids:
        instruments = _group_by(..., lambda row: row.track_id)
        effects = _group_by(..., lambda row: row.track_id)
        links = _group_by(..., lambda row: row.track_id)
    else:
        instruments, effects, links = {}, {}, {}

    patterns = list(db.scalars(
        select(Pattern).where(Pattern.project_id == project_id).order_by(Pattern.pattern_id)
    ).all())
    pattern_ids = [p.pattern_id for p in patterns]
    notes = _group_by(..., lambda row: row.pattern_id) if pattern_ids else {}

    return FullProjectResponse(
        project=project_info(db, project, member),
        tracks=[_full_track(t, instruments, effects, links) for t in tracks],
        patterns=[pattern_service.to_response(p, notes.get(p.pattern_id, [])) for p in patterns],
    )
```

문법 풀이

식	뜻
`db.scalars(select(...)).all()`	여러 행을 읽어 옵니다. `list(...)` 로 감싸 목록으로 만듭니다
`.in_(track_ids)`	트랙 번호 여러 개를 한 번의 조회로 가져옵니다 (`WHERE track_id IN (…)`)
`if track_ids:`	빈 목록으로 `IN ()` 을 보내면 오류가 나므로 트랙이 있을 때만 묻습니다
`instruments.get(track_id, [])`	그 트랙에 장치가 없으면 빈 목록입니다
`lambda row: row.track_id`	무엇으로 묶을지 알려 주는 작은 함수입니다
`pattern_service.to_response(pattern, notes)`	패턴 + 노트 → 응답 한 건 (215). 노트를 넘겨 관계 읽기를 건너뜁니다

4. 왜 이렇게

선택	이유
트랙 · 패턴을 먼저 읽고 장치를 한 꺼번에	트랙마다 서너 번 묻던 것을 조회 여섯 번으로 줄입니다
트랙이 없을 때 조회를 건너뜀	빈 `IN ()` 은 MySQL 이 싫어합니다. MASTER 는 늘 있지만 규칙으로 막아 둡니다
노트를 패턴과 함께 담음	피아노롤과 클립 미리보기가 곧바로 그릴 수 있습니다. 목록 API(217)는 가볍게 두려고 노트를 빼 둡니다
장치 차례를 slot_index 순으로	화면의 이펙트 체인 순서가 곧 `slot_index` 입니다 (FR-263)

선택	이유
없는 표는 담지 않음	clips · markers · automation 은 모델이 없어 조회할 수 없습니다. 응답 스키마(264)가 빈 목록으로 채웁니다

5. 시험

`tests/test_full_project_routes.py` 9건이 모두 통과합니다. `test_full_attaches_devices_to_their_own_track` 에서 Bass Synth 의 악기 · 이펙트가 Keys 의 것과 섞이지 않는지, 연결선이 Keys 에만 붙는지 확인했습니다. `test_full_without_devices_returns_empty_lists` 는 MASTER 한 줄만 있는 새 프로젝트에서도 빈 목록이 나오는지 봅니다. `test_full_viewer_can_read` 는 VIEWER 도 읽을 수 있는지, `test_full_of_someone_elses_project_is_404` · `test_full_unknown_project_is_404` 는 남의 것과 없는 번호를 404 로 막는지 확인합니다.